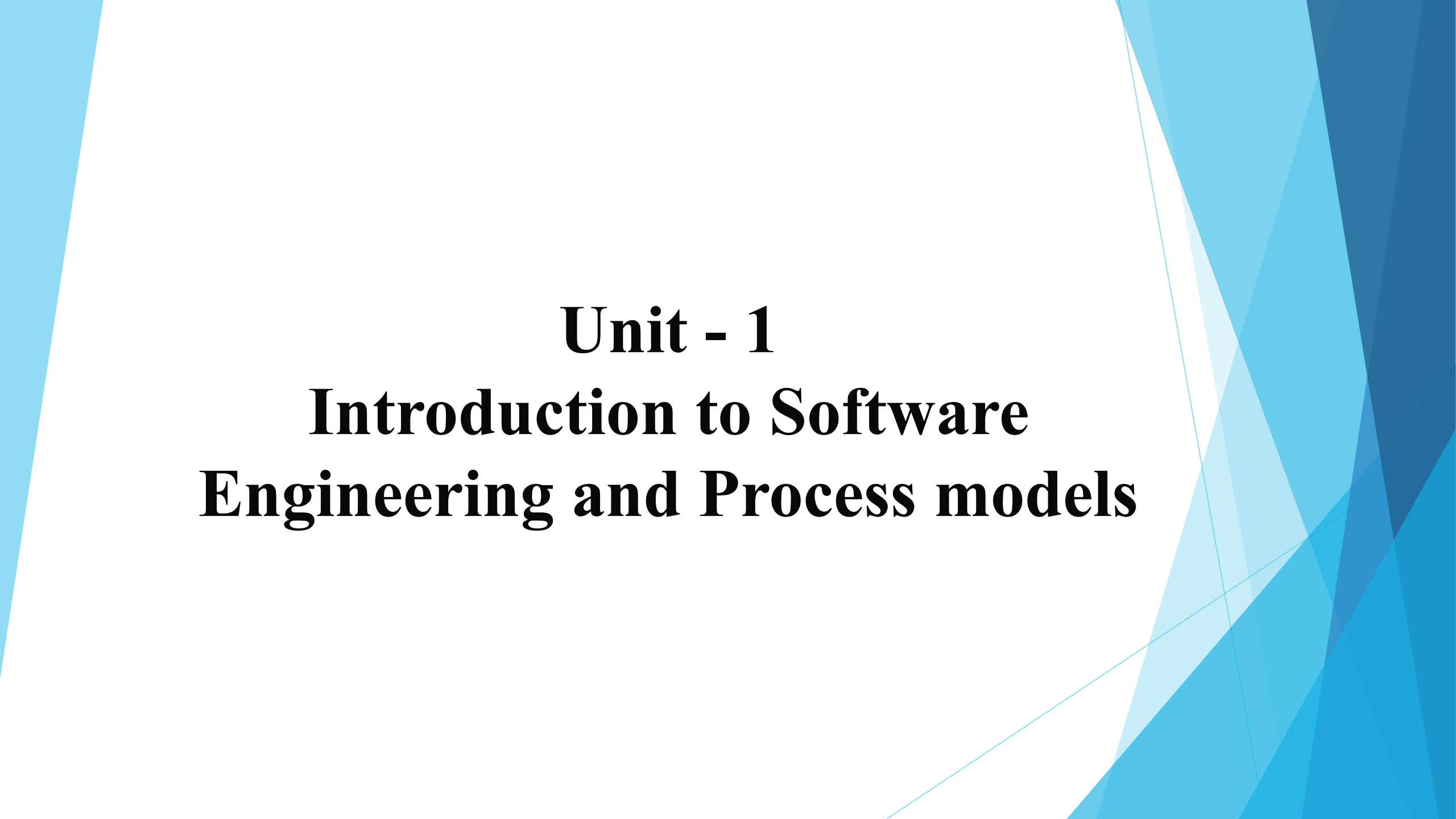


The background features abstract, overlapping geometric shapes in various shades of blue, ranging from light sky blue to deep navy blue, creating a modern, layered effect.

Unit - 1

Introduction to Software Engineering and Process models

Software – A problem solving tool

- ▶ Software is a set of instructions, data or programs used to operate computers and execute specific tasks.
- ▶ **Examples:** Office suites, database programs, web browsers, word processors, software development tools, image editors and communication platforms.
- ▶ **Work Product:** It is computer program that run on specific environment and provide service to end users.

Software Application Domain

1. System software
2. Application software
3. Engineering / scientific software
4. Embedded software
5. Product-line software
6. Web / Mobile application
7. Artificial intelligence software

Problems with Software

- ▶ Why does it take so long to get software finished?
- ▶ Why are development costs so high?
- ▶ Why can't we find all errors before we give the software to our customers?
- ▶ Why do we spend so much time and effort maintaining existing programs?
- ▶ Why do we continue to have difficulty in measuring progress as software is being developed and maintained?

Why study Software engineering?

- ▶ Software has become deeply embedded in virtually every aspect of our lives, concerted effort should be made to understand the problem before a software solution is developed.
- ▶ Large teams of people now create computer programs that were once built by a single individual. Sophisticated software that was once implemented in a predictable, self-contained, computing environment is now embedded inside everything from consumer electronics to medical devices to weapons systems. It follows that design becomes a pivotal activity.
- ▶ Individuals, businesses, and governments increasingly rely on software for strategic and tactical decision making as well as day-to-day operations and control. If the software fails, people and major enterprises can experience anything from minor inconvenience to catastrophic failures. It follows that software should exhibit high quality.

Why study Software engineering ? contd.

- ▶ As the perceived value of a specific application grows, the likelihood is that its user base and longevity will also grow. As its user base and time-in-use increase, demands for adaptation and enhancement will also grow. It follows that software should be maintainable.
- ▶ Conclusion: software in all of its forms and across all of its application domains should be engineered. And that leads us to the subject— **Software Engineering**.

Software Engineering – Evolution of software product

- ▶ It is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.
- ▶ It includes the development and building of computer systems software and application software.

Software Engineering – A layered technology.



- ▶ **Quality focus layer** – a bedrock that supports software engineering
- ▶ **Process layer** – a foundation of software engineering that enables rational and timely development of software
- ▶ **Methods** – provide the technical how-to's for building software.
- ▶ **Tools** – Automated or semi-automated support for the process and methods.

Software process

- ▶ A process is a collection of activities, actions, and tasks that are performed when some work product is to be created.
- ▶ It is an adaptable approach that enables software team to choose appropriate set of actions and tasks.
- ▶ Intent is to deliver the software in a timely manner.

Process framework

- ▶ It establishes the foundation for a complete software engineering process by identifying a small number of framework activities that are applicable to all software projects, regardless of their size or complexity.
- ▶ It encompasses five activities: Communication, Planning, Modelling, Construction and Deployment.
- ▶ These five generic framework activities can be used during the development of small, simple programs, the creation of Web applications and for the engineering of large, complex computer-based systems.

Process framework activities

1. **Communication:** Communicate and collaborate with the customer to understand stakeholders' objectives for the project and to gather requirements that help define software features and functions.
2. **Planning:** Describe the technical tasks to be conducted, the risks that are likely, the resources that will be required, the work products to be produced, and a work schedule.
3. **Modelling:** Create models to better understand software requirements and the design that will achieve those requirements.
4. **Construction:** Code generation and the testing that is required to uncover errors in the code.perform testing
5. **Deployment:** Software delivery to customer and customer evaluate it and give feedback.

Umbrella activities

- ▶ Umbrella activities are applied throughout a software project.
- ▶ It helps a software team to manage and control progress, quality, change, and risk.
- ▶ Typical umbrella activities include: Software project tracking and control, Risk management, Software quality assurance, Technical reviews, reusability management and work product preparation & production

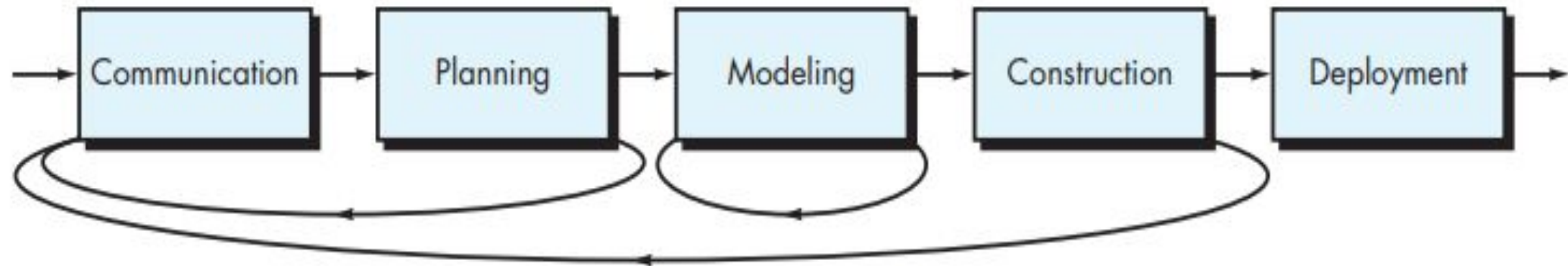
Process Flow

- ▶ Process flow—describes how the framework activities and the actions and tasks that occur within each framework activity are organized with respect to sequence and time
- ▶ **A linear process flow** executes each of the five framework activities in sequence, beginning with communication and culminating with deployment
- ▶ **An iterative process flow** repeats one or more of the activities before proceeding to the next
- ▶ **An evolutionary process flow** executes the activities in a “circular” manner.
- ▶ **A parallel process flow** executes one or more activities in parallel with other activities (e.g. modeling for one aspect of the software might be executed in parallel with construction of another aspect of the software).

Process flow

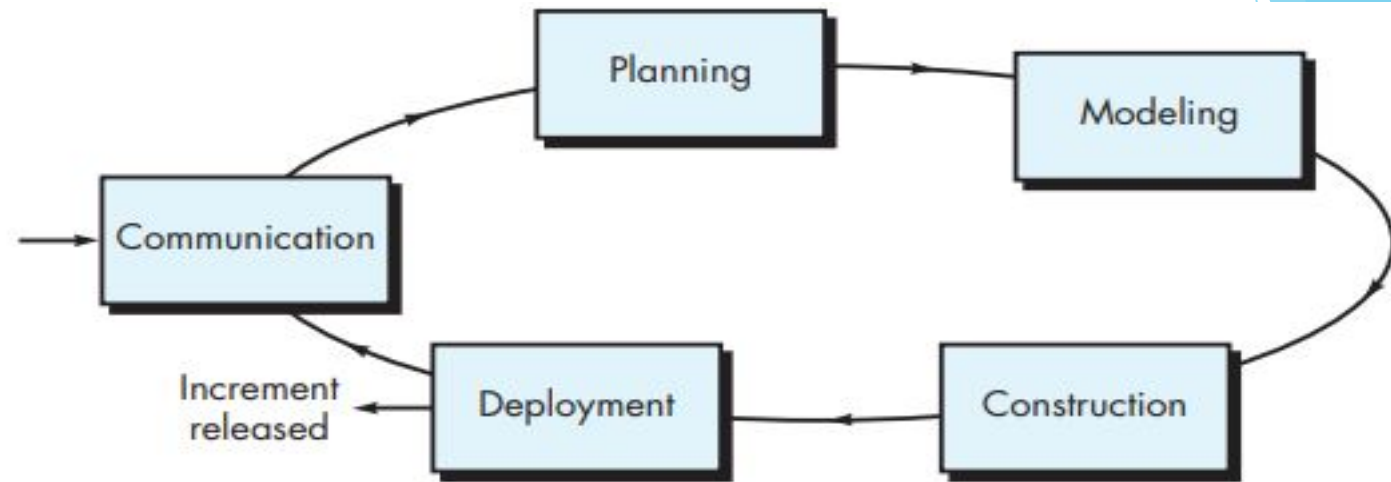


(a) Linear process flow

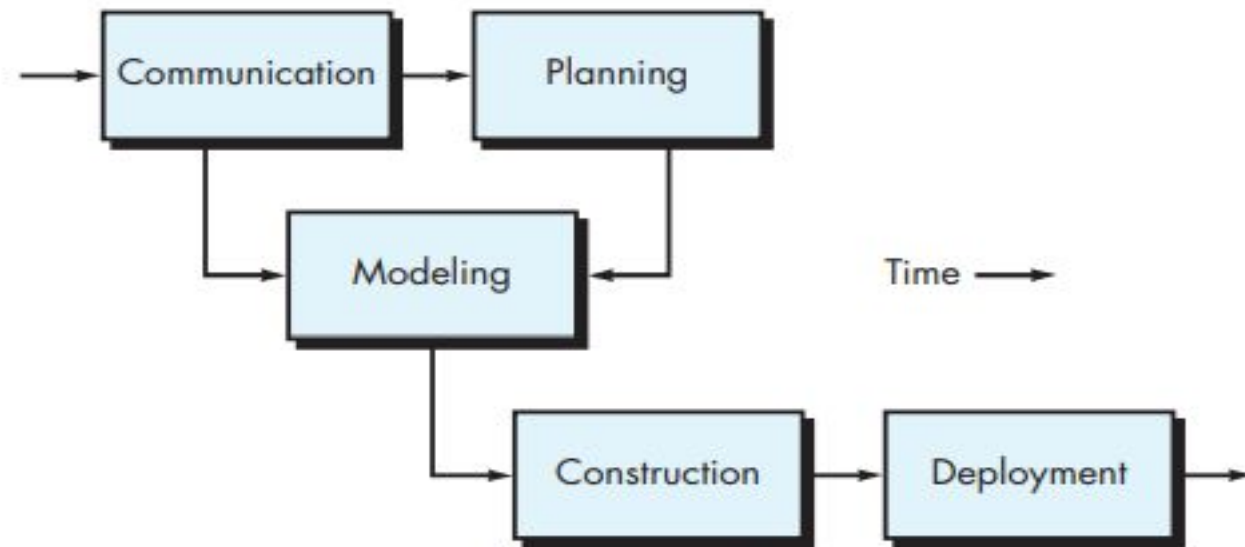


(b) Iterative process flow

Process flow



(c) Evolutionary process flow



(d) Parallel process flow

Capability Maturity Model (CMM)

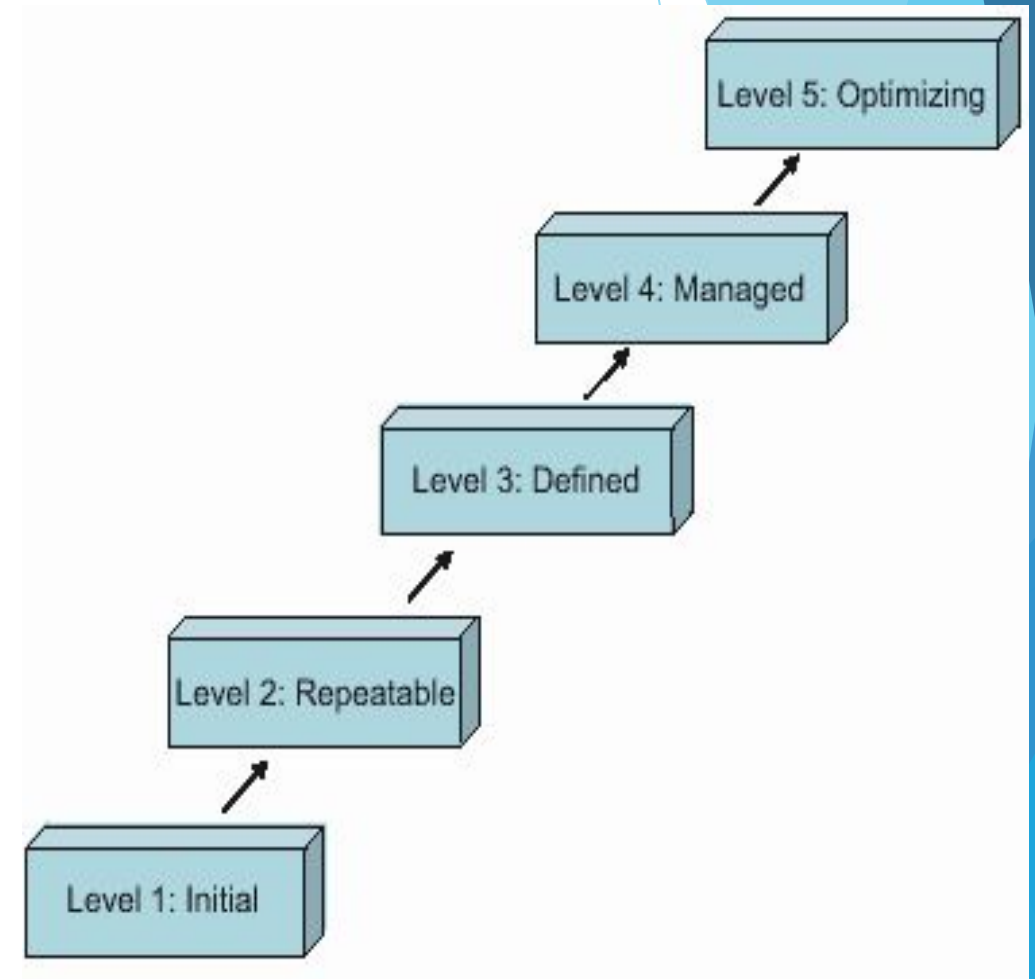
- ▶ CMM was developed by the Software Engineering Institute (SEI) at Carnegie Mellon University in 1987.
- ▶ It is a framework which is used to analyze the approach and techniques followed by any organization to develop software products.
- ▶ It provides guidelines to develop software product but it is not a process model.
- ▶ This model describes a strategy for software process improvement that should be followed by moving through 5 different levels.

The Five Maturity Levels of CMM

- ▶ A maturity level is a well-defined evolutionary plateau toward achieving a mature software process.
- ▶ These five levels define a scale to measure the maturity of software process and to evaluate the capability of these processes.
- ▶ Each maturity level consist of a set of process goals

The Five Maturity Levels of CMM

1. **Initial:** Processes are unpredictable, poorly controlled and reactive.
2. **Repeatable:** Processes are planned, documented, performed, monitored and controlled at project level.
3. **Defined:** Processes are well characterized and understood. Processes, procedure and tools are defined at organization level.



The Five Maturity Levels of CMM

- 4. **Managed:** Processes are controlled using statistical and other quantitative techniques.
- 5. **Optimizing:** Process performance is improved through incremental and innovative technological improvement.

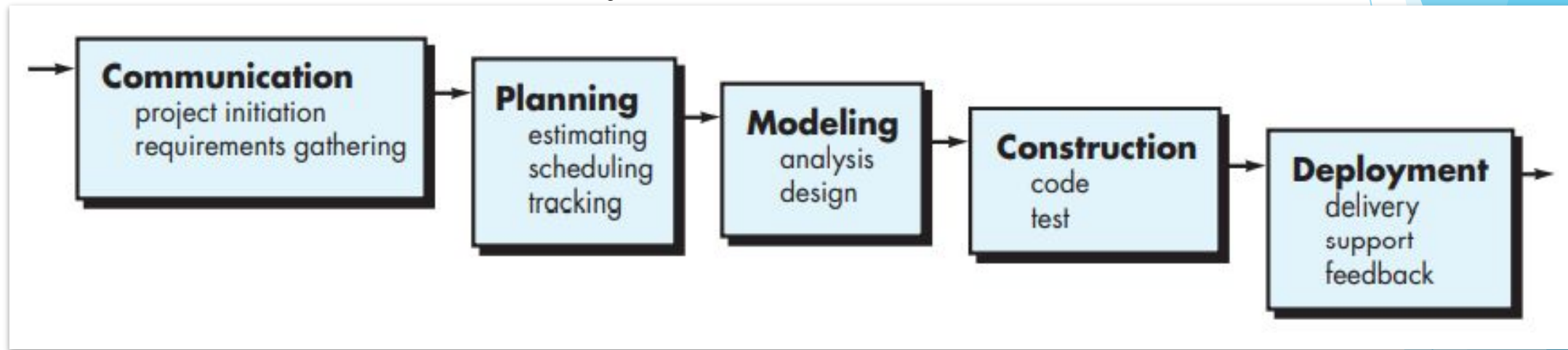


Prescriptive Process Models

- ▶ A process model provides a specific roadmap for software engineering work.
- ▶ Prescriptive process models are sometimes referred to as “traditional” process models.
- ▶ It defines the flow of all activities, actions and tasks, the degree of iteration, the work products, and the organization of the work that must be done.
- ▶ A prescriptive process model strives for structure and order in software development. Activities and tasks occur sequentially with defined guidelines for progress.

1. Waterfall Model

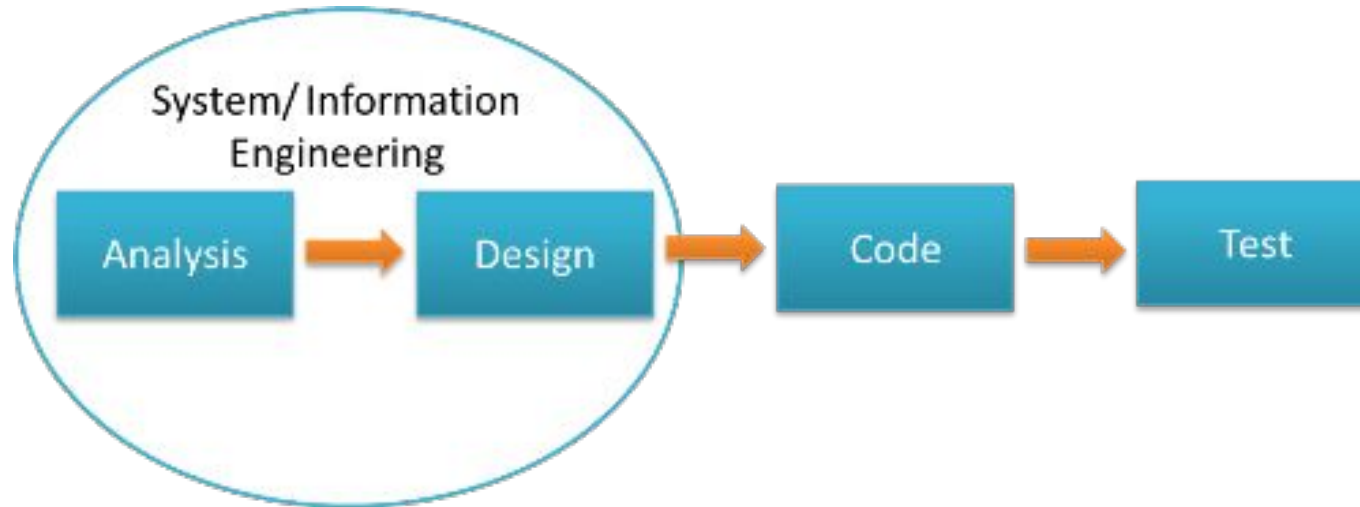
- ▶ The waterfall model is also called as '**Linear sequential model**' or '**Classic life cycle model**'.



- ▶ This model suggests a systematic, sequential approach to software development that begins with customer specification of requirements and progresses through planning, modeling, construction, and deployment, culminating in ongoing support of the completed software.

1. Waterfall Model

- ▶ Alternative representation of this model is:



- ▶ System engineering and analysis encompass requirements gathering at the system level with a small amount of top level design and analysis. Information engineering encompasses requirements gathering at the strategic business level and at the business area level.

1. Waterfall Model

Advantages:

- It is simple and easy to understand, implement, and use.
- All the requirements are known at the beginning of the project, hence it is easy to manage.
- It avoids overlapping of phases because each phase is completed at once.
- This model works for small projects because the requirements are understood very well.
- This model is preferred for those projects where the quality is more important as compared to the cost of the project.

1. Waterfall Model

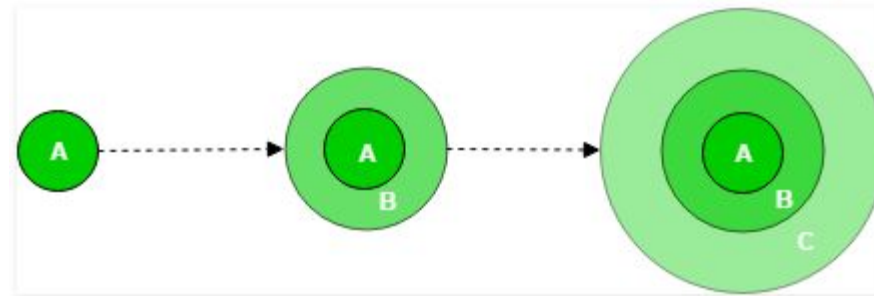
Disadvantages:

- This model is not good for complex and object oriented projects.
- It is a poor model for long projects.
- The problems with this model are uncovered, until the software testing.
- The amount of risk is high.

The linear sequential model is the oldest and the most widely used paradigm for software engineering.

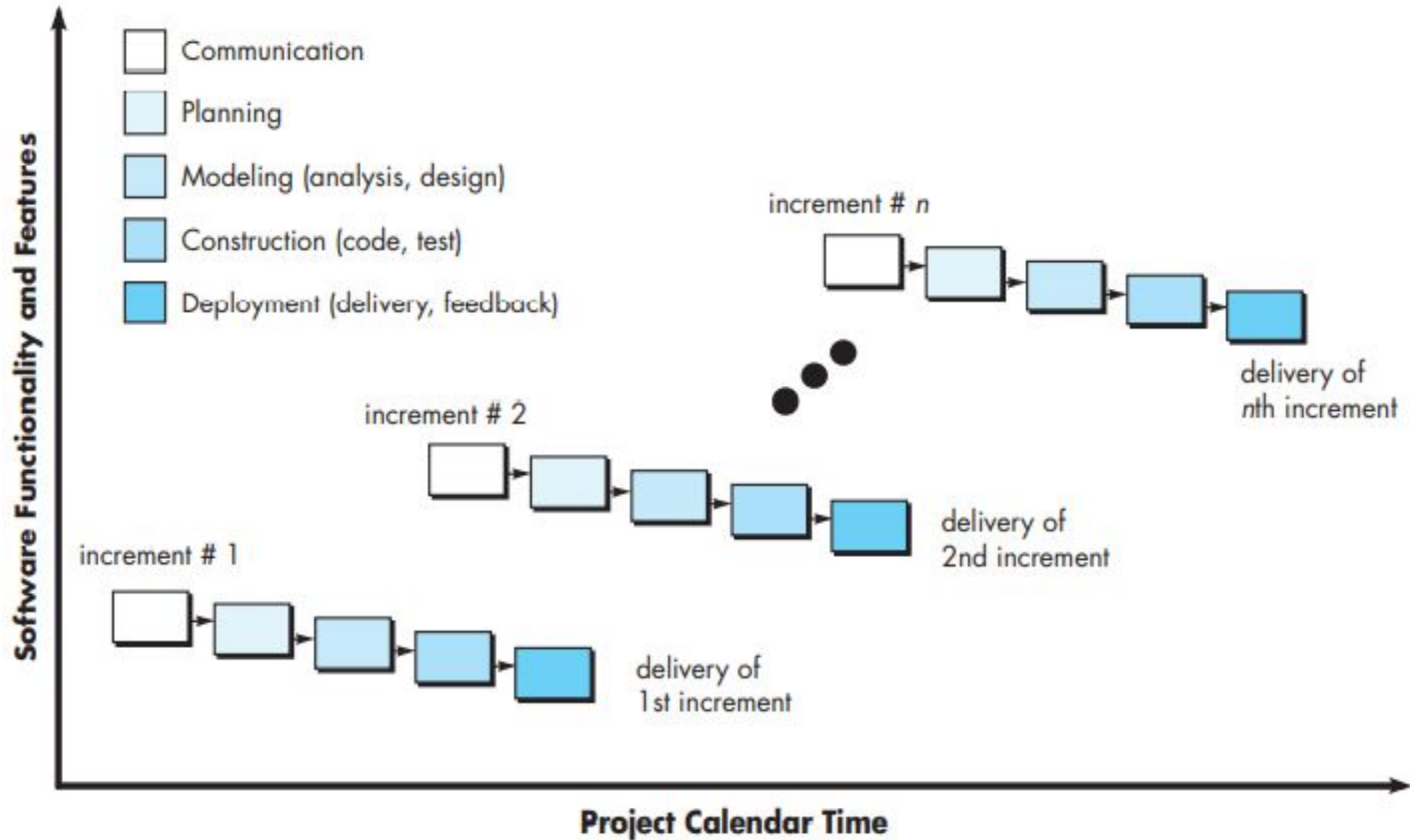
2. Incremental Process Model

- ▶ The incremental model delivers a series of releases, called increments, that provide progressively more functionality for the customer as each increment is delivered.
- ▶ It is also known as Successive version model.



- ▶ A, B, C are modules of Software Product that are incrementally developed and delivered.

3. Incremental Process Model



3. Incremental Process Model

- ▶ When an incremental model is used, the first increment is often a core product. That is, basic requirements are addressed but many supplementary features remain undelivered.
- ▶ The core product is used by the customer and undergoes detailed evaluation. As a result of use and evaluation, a plan is developed for the next increment.
- ▶ The plan addresses the modification of the core product to better meet the customer's need and the delivery of additional features and functionality.
- ▶ This process is repeated following the delivery of each increment, until the complete product is produced.

3. Incremental Process Model

This process model is used when :

- Requirements of the system are clearly understood
- When demand for an early release of a product arises
- When software engineering team are not very well skilled or trained
- When high-risk features and goals are involved

3. Incremental Process Models

Advantages:

- The software will be generated quickly during the software life cycle
- It is flexible and less expensive to change requirements and scope
- Throughout the development stages changes can be done
- A customer can respond to each building.

3. Incremental Process Models

Disadvantages:

- Requires good planning and design.
- Well defined module interfaces are required.
- Total Cost is high.

Evolutionary Process model

- ▶ Evolutionary model is a combination of Iterative and Incremental model of software development life cycle.
- ▶ Evolutionary models are iterative type models.
- ▶ They allow to develop more complete versions of the software.
- ▶ It is normally useful for very large products, where it is easier to find modules for incremental implementation.

Evolutionary Process model

Following are the evolutionary process models.

1. The spiral model

1. Spiral model

- ▶ Originally proposed by Barry Boehm, the spiral model is an evolutionary software process model that couples the iterative nature of prototyping with the controlled and systematic aspects of the waterfall model.
- ▶ It provides the potential for rapid development of increasingly more complete versions of the software.
- ▶ Using the spiral model, software is developed in a series of evolutionary releases. During early iterations, the release might be a model or prototype. During later iterations, increasingly more complete versions of the engineered system are produced.

Spiral model

- ▶ The spiral model has five phases. A software project repeatedly passes through these phases in iterations called Spirals.

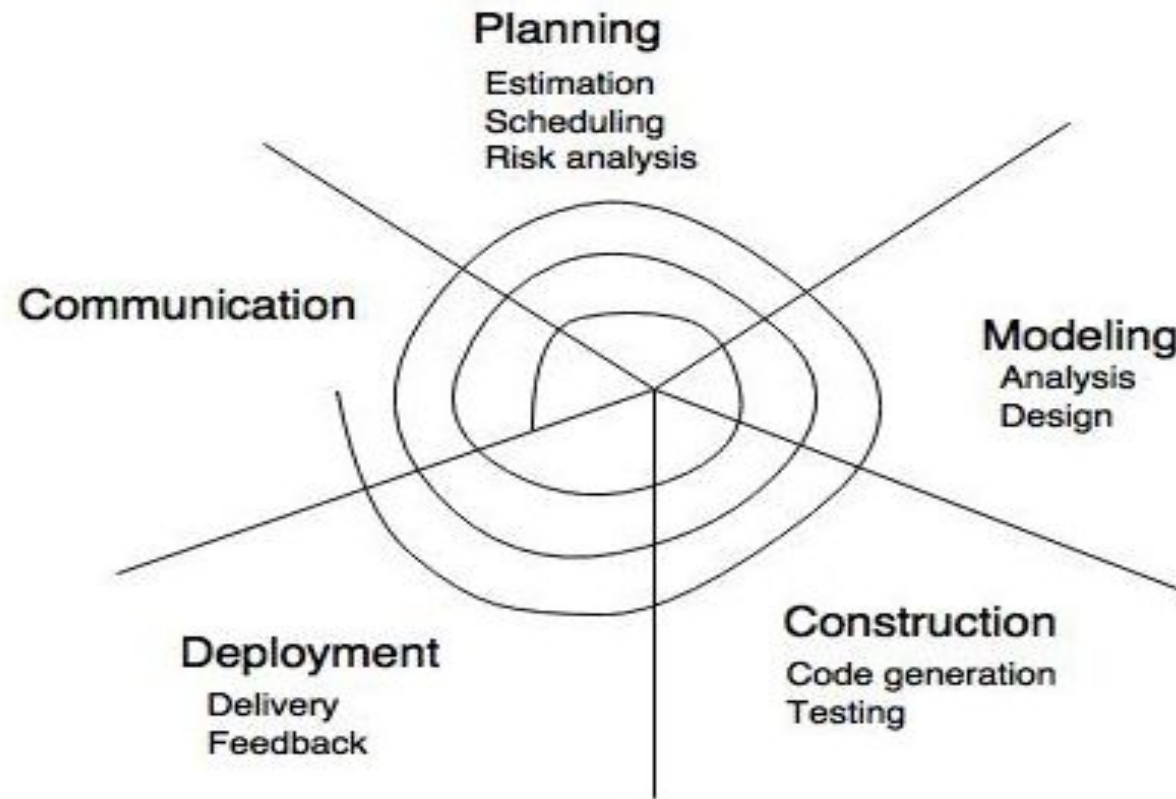


Fig. - The Spiral Model

Spiral model

- ▶ A spiral model is divided into a set of framework activities defined by the software engineering team. Each framework activities represent one segment of the spiral path.
- ▶ It provides support for Risk Handling.
- ▶ The Spiral model is called as a Meta Model because it subsumes all the other SDLC models.
- ▶ It is a realistic approach to the development of large-scale systems and software.

Spiral model

Spiral Model is used when:

- When deliverance is required to be frequent.
- When the project is large and have high budget
- When requirements are unclear and complex
- When changes may require at any time

Spiral model

Advantages:

- It reduces high amount of risk.
- It is good for large and critical projects.
- It gives strong approval and documentation control.
- In spiral model, the software is produced early in the life cycle process.

Disadvantages:

- It can be costly to develop a software model.
- It is not used for small projects.

Agility

- ▶ Agility is the ability to both create and respond to change in order to profit in a turbulent business environment.
- ▶ Agility is dynamic, content specific, aggressively change embracing, and growth oriented.
- ▶ It helps software team provide quick and unpredictable responses to the feedback that they receive on their project.

Agile Process

- ▶ Agile process refers to a software development approach based on iterative development.
- ▶ Agile methods break tasks into smaller iterations or parts do not directly involve long term planning.
- ▶ Each iteration is considered as a short time "frame" in the Agile process model, which typically lasts from one to four weeks.
- ▶ The division of the entire project into smaller parts helps to minimize the project risk and to reduce the overall project delivery time requirements.

Agile Process

- ▶ An agile process reduces the cost of change because software is released in increments and change can be better controlled within an increment.



Example-agile development

In traditional Waterfall model -

- At a high level, the project teams would spend 15% of their time on gathering requirements and analysis (1.5 months)

- 20% of their time on design (2 months)

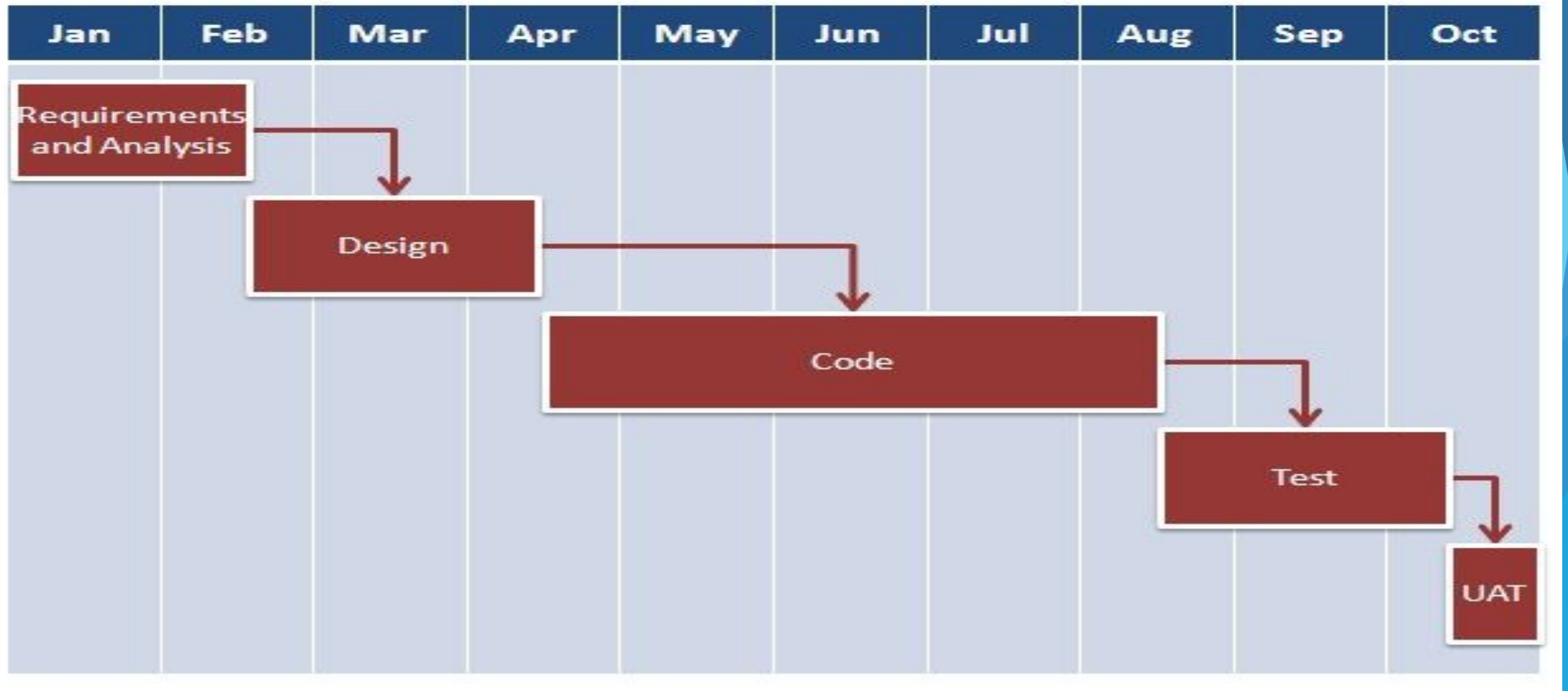
- 40% on coding (4 months) and unit testing

- 20% on System and Integration testing (2 months).

At the end of this cycle, the project may also have 2 weeks of User Acceptance testing by marketing teams.

In this approach, the customer does not get to see the end product until the end of the project, when it becomes too late to make significant changes.

Example

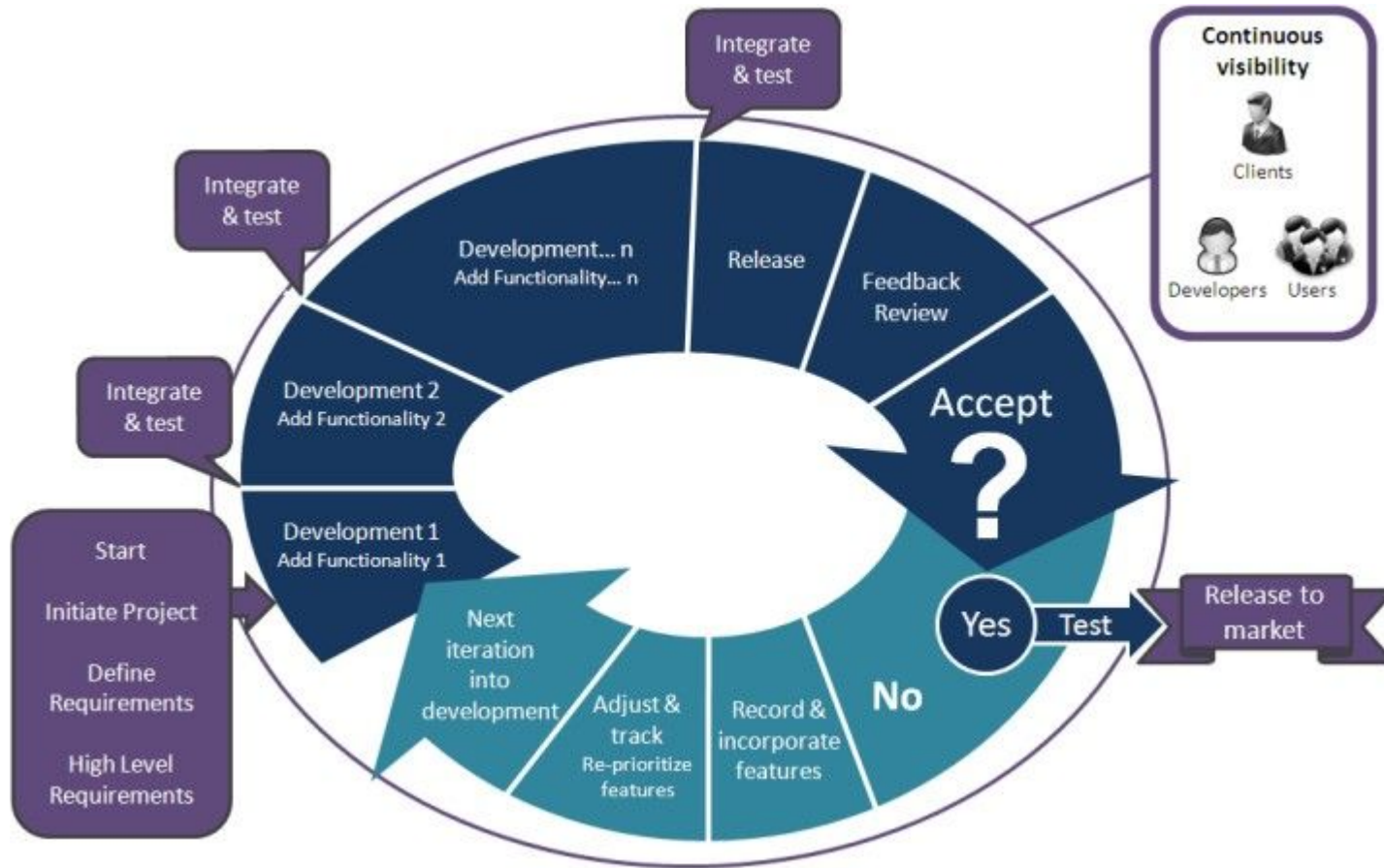


Example-agile development

With **Agile development** methodology -

- Each project is broken up into several 'Iterations'.
- All Iterations should be of the same time duration (between 2 to 8 weeks).
- At the end of each iteration, a working product should be delivered.
- In simple terms, in the Agile approach the project will be broken up into 10 releases (assuming each iteration is set to last 4 weeks).
- Rather than spending 1.5 months on requirements gathering, in Agile software development, the team will decide the basic core features that are required in the product and decide which of these features can be developed in the first iteration.
- Any remaining features that cannot be delivered in the first iteration will be taken up in the next iteration or subsequent iterations, based on priority.
- At the end of the first iterations, the team will deliver a working software with the features that were finalized for that iteration.
- There will be 10 iterations and at the end of each iteration the customer is delivered a working software that is incrementally enhanced and updated with the features that were shortlisted for that iteration.

Agile Development



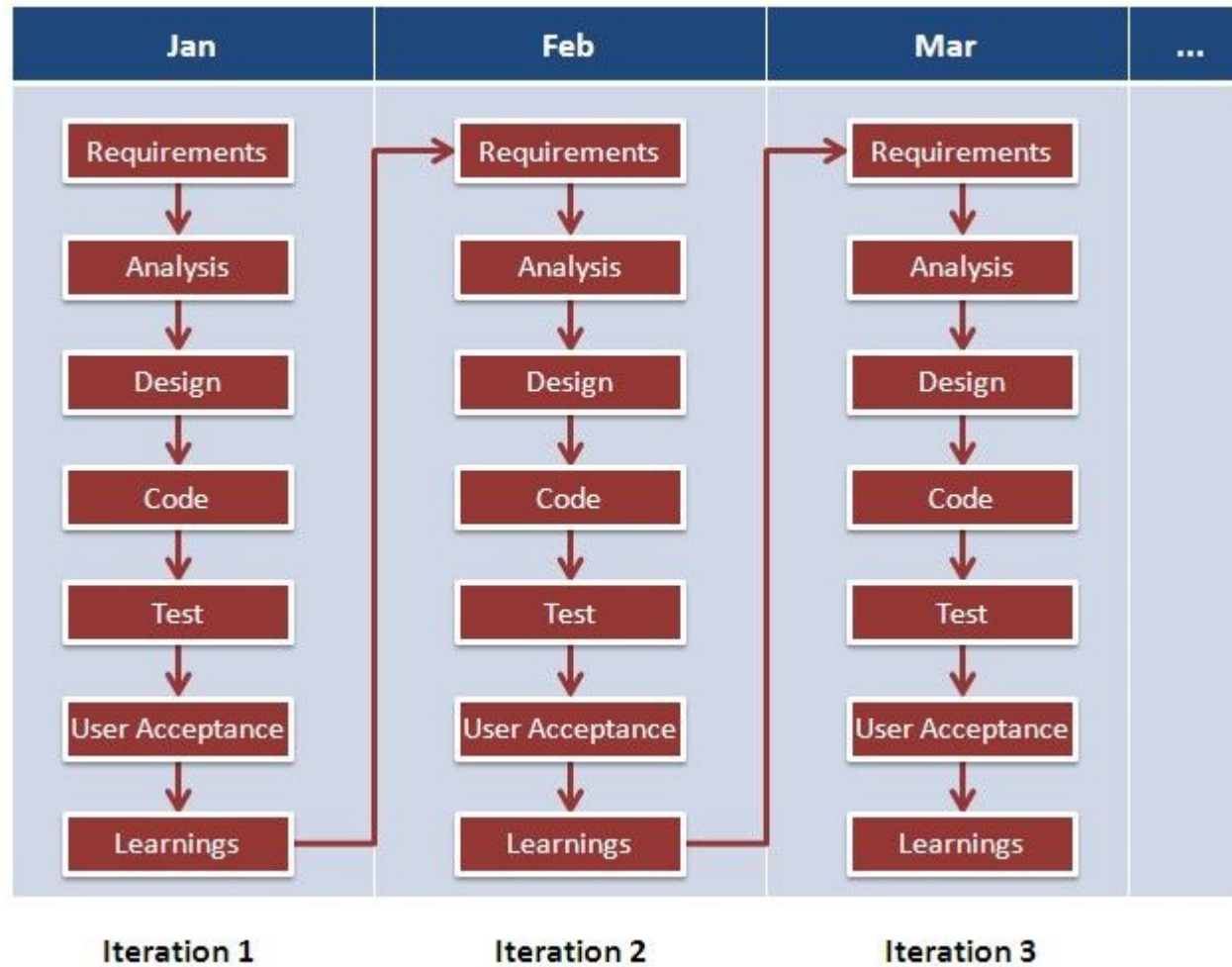
Agile Process Model phases

1. **Requirements gathering:** In this phase, you must define the requirements, explain business opportunities, plan the time and effort needed to build the project.
2. **Design the requirements:** In this, You can use the user flow diagram or the high-level UML diagram to show the new features and show how it will apply to your existing system.
3. **Construction:** When the team defines the requirements, the work begins. Designers and developers start working on their project, which aims to deploy a working product.

Agile Process Model phases

4. **Testing:** In this, the Quality Assurance team examines the product's performance and looks for the bug.
5. **Deployment:** In this phase, the team issues a product for the user's work environment.
6. **Feedback:** After releasing the product, the last step is feedback. In this, the team receives feedback about the product and works through the feedback.

Agile Development



Agile Process Model

Agile Model can be used when:

- When frequent changes are required.
- When a highly qualified and experienced team is available.
- When a customer is ready to have a meeting with a software team all the time.

Agile Process Model

Benefits of using agile model is summarized as follows:

1. Frequent Delivery
2. Face-to-Face Communication with clients.
3. Efficient design and fulfils the business requirement.
4. Anytime changes are acceptable.
5. It reduces total development time.
6. It allows for a greater level of control around the software that is delivered.

Agile principles

1. Satisfy the customer through early and continuous delivery
2. Welcome changing requirements, even late in development
3. Deliver working software frequently
4. Business people and developers must work together daily throughout the project
5. Build projects around motivated individuals
6. Face-to-face conversations

Agile principles

7. Working software is the primary measure of progress
8. Agile processes promote sustainable development
9. Continuous attention to technical excellence and good design enhances agility
10. Simplicity
11. Self-organizing teams
12. Adjust ways of working in regular intervals to be more effective

Agile Development

- ▶ Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
- ▶ Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
- ▶ Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
- ▶ Business people and developers must work together daily throughout the project.
- ▶ Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
- ▶ The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.

Agile Development

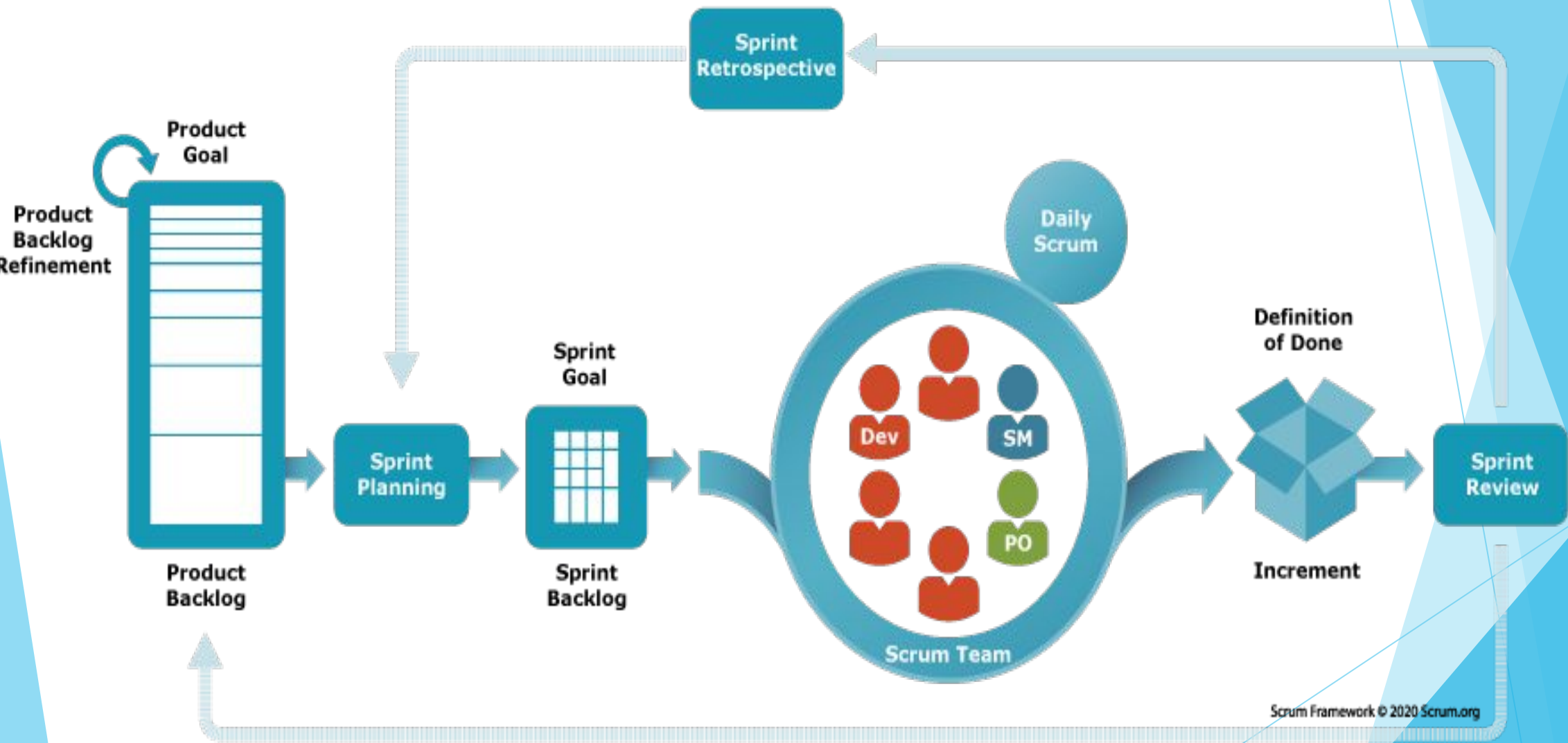
- ▶ Working software is the primary measure of progress.
- ▶ Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
- ▶ Continuous attention to technical excellence and good design enhances agility.
- ▶ Simplicity - the art of maximizing the amount of work not done - is essential.
- ▶ The best architectures, requirements, and designs emerge from self-organizing teams.
- ▶ At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

Agile testing methods

- Scrum
- Crystal
- Dynamic Software Development Method(DSDM)
- Feature Driven Development(FDD)
- Lean Software Development
- eXtreme Programming(XP)

Scrum

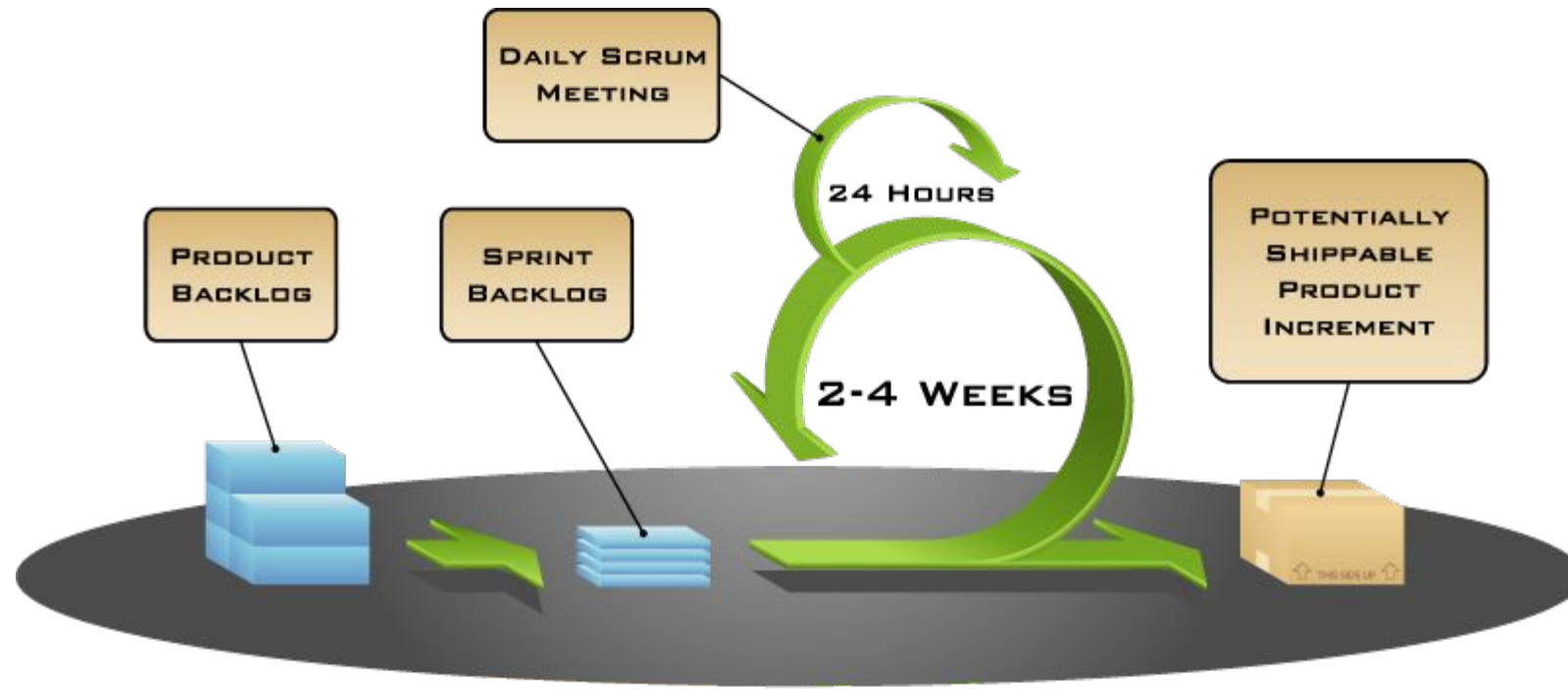
- ▶ Scrum is a project management framework with aggressive deadlines, complex requirements and a degree of uniqueness.
- ▶ It is the most popular agile framework, which concentrates particularly on how to manage tasks within a team-based development environment.
- ▶ In Scrum, projects move forward via a series of iterations called sprints.
- ▶ Each sprint is typically two to four weeks long.
- ▶ Scrum is relatively simple to implement and focuses on quick and frequent deliveries.



SCRUM

- ▶ SCRUM is an agile, lightweight process for managing and controlling software and product development in rapidly changing environments.
- ▶ Iterative, incremental process
- ▶ Team-based approach developing systems/ products with rapidly changing requirements
- ▶ Controls the chaos of conflicting interest and needs
- ▶ Improve communication and maximize cooperation
- ▶ Protecting the team from disruptions and impediments
- ▶ A way to maximize productivity

Functionality of Scrum



COPYRIGHT © 2005, MOUNTAIN GOAT SOFTWARE

Components of Scrum

- ▶ Scrum Roles
- ▶ The Process
- ▶ Scrum Artifacts

Scrum Roles

The Product Owner

- ▶ Owns definition of success
- ▶ Manages ROI through prioritization and release plans
- ▶ Knows what needs to be build and in what sequence this should be done
- ▶ Typically a product manager

The Scrum Master

Owns the process

Teaches the Product Owner and the Team

The Team

Owns the production and engineering process

Scrum Team

- ▶ Typically 5-10 people
- ▶ Cross-functional (QA, Programmers, UI Designers, etc.)
- ▶ Members should be full-time
- ▶ Team is self-organizing
- ▶ Membership can change only between sprints

The Process

- ▶ Sprint Planning Meeting
- ▶ Sprint
- ▶ Daily Scrum
- ▶ Sprint Review Meeting

Sprint Planning Meeting

- ▶ Product Owner describes highest priority features to the Team.
- ▶ Team decides what they can commit to delivering in the Sprint.
- ▶ A collaborative meeting in the beginning of each Sprint between the Product Owner, the Scrum Master and the Team
- ▶ Takes 8 hours and consists of 2 parts (“before lunch and after lunch”)

Sprint Planning Meeting

- ▶ 1st Part:
 - ▶ Creating Product Backlog
 - ▶ Determining the Sprint Goal.
 - ▶ Participants: Product Owner, Scrum Master, Scrum Team
- ▶ 2nd Part:
 - ▶ Participants: Scrum Master, Scrum Team
 - ▶ Creating Sprint Backlog

Sprint

- ▶ A month-long iteration, during which is incremented a product functionality
- ▶ NO outside influence can interference with the Scrum team during the Sprint
- ▶ Each Sprint begins with the Daily Scrum Meeting

Daily Scrum

- ▶ Is a short (15 minutes long) meeting, which is held every day before the Team starts working
- ▶ Participants: Scrum Master (chairman), Scrum Team
- ▶ Every Team member should answer on 3 questions
 - ▶ What did you do since the last Scrum?
 - ▶ What are you doing until the next Scrum?
 - ▶ What is stopping you getting on with the work?

Sprint Review Meetings

- ▶ Is held at the end of each Sprint
- ▶ Business functionality which was created during the Sprint is demonstrated to the Product Owner
- ▶ Informal, should not distract Team members of doing their work

Scrum Artifacts

The Product Backlog

The Sprint Backlog

The Sprint Burndown Chart

The Product Increment

Product Backlog

	Item #	Description	Est	By
Very High				
	1	Finish database versioning	16	KH
	2	Get rid of unneeded shared Java in database	8	KH
		- Add licensing	-	-
	3	Concurrent user licensing	16	TG
	4	Demo / Eval licensing	16	TG
		Analysis Manager		
	5	File formats we support are out of date	160	TG
	6	Round-trip Analyses	250	MC
High				
		- Enforce unique names	-	-
	7	In main application	24	KH
	8	In import	24	AM
		- Admin Program	-	-
	9	Delete users	4	JM
		- Analysis Manager	-	-
	10	When items are removed from an analysis, they should show up again in the pick list in lower 1/2 of the analysis tab	8	TG
		- Query	-	-
	11	Support for wildcards when searching	16	T&A
	12	Sorting of number attributes to handle negative numbers	16	T&A
	13	Horizontal scrolling	12	T&A
		- Population Genetics	-	-
	14	Frequency Manager	400	T&M
	15	Query Tool	400	T&M
	16	Additional Editors (which ones)	240	T&M
	17	Study Variable Manager	240	T&M
	18	Haplotypes	320	T&M
	19	Add icons for v1.1 or 2.0	-	-
		- Pedigree Manager	-	-
	20	Validate Derived kindred	4	KH
Medium				
		- Explorer	-	-
	21	Launch tab synchronization (only show queries/analyses for logged in users)	8	T&A
	22	Delete settings (?)	4	T&A

Kanban-work flow management

Kanban is a highly visual workflow management method that is popular among Lean teams.

Used to visualize and actively manage the creation of products with an emphasis on continual delivery, while not overburdening the development team.

Like Scrum, Kanban is a process designed to help teams work together more effectively.

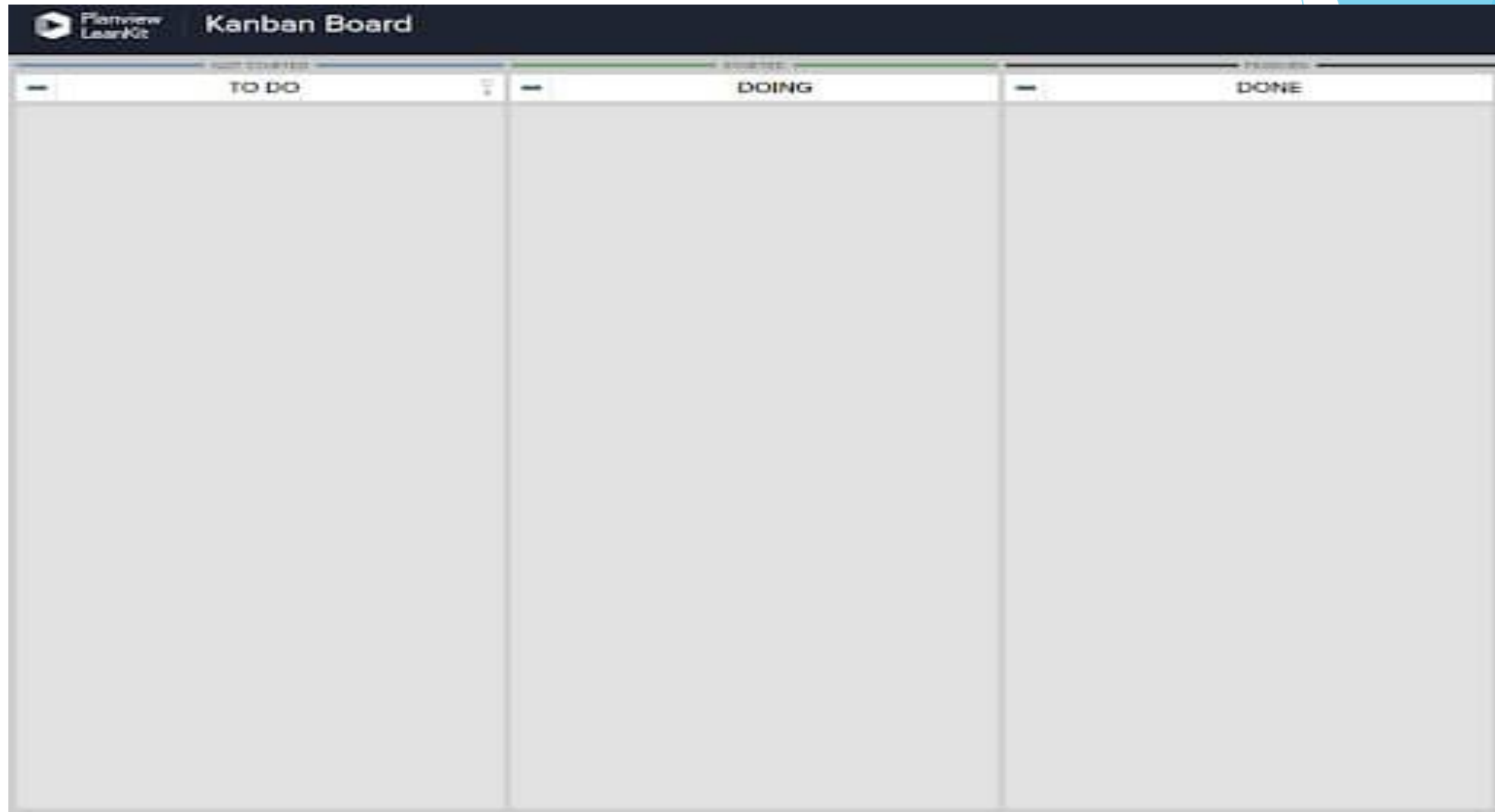
Kanban is based on 3 basic principles:

- Visualize what you'll do today (workflow): Seeing all the items within the context of each other can be very informative

- Limit the amount of work in progress (WIP): This helps balance the flow-based approach so teams don't start and commit to too much work at once

- Enhance flow: When something is finished, the next highest priority item from the backlog is pulled into play

Kanban-workflow management



Kanban

This framework used to implement [agile](#) and [DevOps](#) software development.

It requires real-time communication of capacity and full transparency of work.

Work items are represented visually on a kanban board, allowing team members to see the state of every piece of work at any time.

Cards on virtual kanban boards will often also feature screenshots and other technical details that is valuable to the assignee. Allowing team members to see the state of every work item at any given point in time, as well as all of the associated details, ensures increased focus, full traceability, and fast identification of blockers and dependencies.

Advanced Trends in Software Engineering

- ▶ **Block chain**

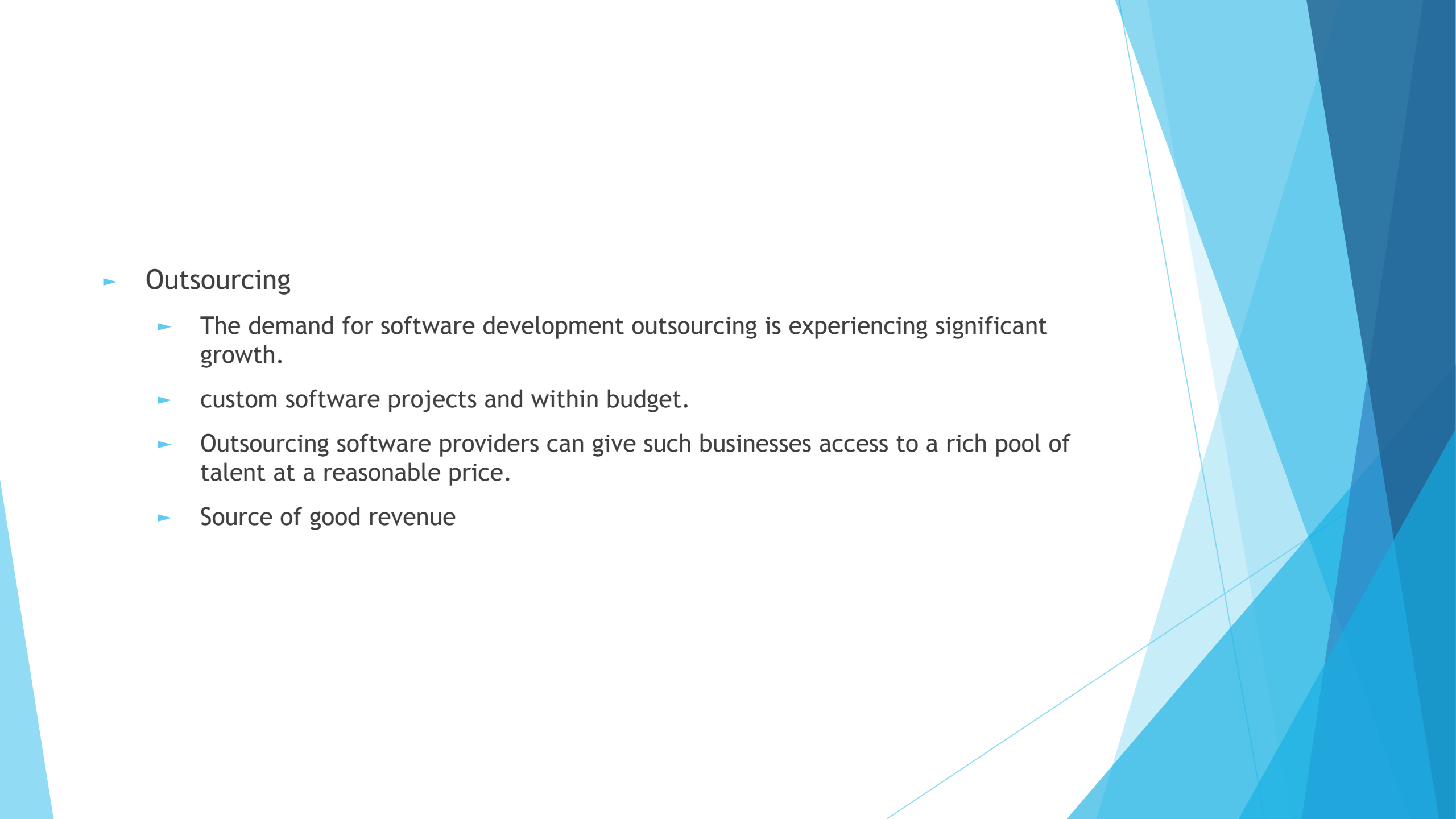
- ▶ As everything is going around decentralized and distributed environment
- ▶ System demands/expects an exceptional level of security and transparency for various transactions, and can be applicable to many business sectors



Cloud computing Facilitates collaboration

- ▶ Demand for cloud services and tools has grown due to the necessity to facilitate remote work. Companies have had to implement the appropriate infrastructure and support employees in creating a work environment in their home offices
- ▶ There are software products and services hosted in public, private, or hybrid clouds.
- ▶ They are infrastructure-agnostic and can run on multiple servers, so any company can easily adopt them.
- ▶ Based on containers and microservices, cloud-native software solutions provide flexibility and scalability, and improve the productivity of software development teams.

- ▶ Infrastructure as a Code(IaC) and DevOps
 - ▶ Modern application development environments are intricate, with all the clusters, microservices, containers, APIs, serverless functions, and interdependencies that developers should keep track of.
 - ▶ DevOps teams are turning to IaC — a method of using code to manage, automate, configure, and drive continuous improvement on dynamic computing resources. Traditionally, organizations relied on a few highly skilled individuals who understood what their code depended on and how it was delivered, which resulted in bottlenecks and dependencies.
 - ▶ Using IaC, the DevOps team can create a single, trusted code base for an organization that will be used to track that apps are deployed according to security standards.
 - ▶ Implementing IaC also allows teams to track inconsistencies in code, making it easier to fix vulnerabilities. This approach makes it possible to implement the best configuration practices within the organization's security policies and rules.

The background of the slide features abstract, overlapping geometric shapes in various shades of blue, ranging from light sky blue to deep navy blue. These shapes are primarily located on the right side of the slide, creating a modern, dynamic feel.

▶ Outsourcing

- ▶ The demand for software development outsourcing is experiencing significant growth.
- ▶ custom software projects and within budget.
- ▶ Outsourcing software providers can give such businesses access to a rich pool of talent at a reasonable price.
- ▶ Source of good revenue

- ▶ LCNC(Low Code No code)
 - ▶ Simplified SE approach
 - ▶ LCNC platforms enable practically anyone to develop software without traditional coding via graphical user interfaces and configuration. Such programming environments have great potential because they allow people to implement their ideas into digital solutions with limited or no software development skills
- ▶ React Native
 - ▶ Tools for Mobile developers
 - ▶ Tool for hybrid development since this is a painless method to create apps with rich functionality and excellent performance for multiple platforms using a single code base.

▶

- ▶ Progressive Web Applications

- ▶ If you think about platform-specific apps and web apps in terms of capabilities and reach, platform-specific apps represent the best of capabilities whereas web apps represent the best of reach. So where do Progressive Web Apps fit in?
- ▶ Progressive Web Apps (PWA) are built and enhanced with modern APIs to deliver enhanced capabilities, reliability, and installability while reaching *anyone, anywhere, on any device* with a single codebase.

- ▶ IoT

- ▶ edge computing.
- ▶ Over the past year, more organizations have been moving their analytics and decision-making from centralized cloud platforms to the edge, i.e., closer to the data source.

▶ 5G Technology

- ▶ 5G connectivity implementation is among the top trends for 2023.
- ▶ The major reason for this is that the technology helps companies realize the full potential of IoT ecosystems. The more reliable the connectivity is, the better the performance demonstrated by connected devices.

▶ Augmented Reality

- ▶ The demand for online shopping has increased significantly.
- ▶ With AR, eCommerce mobile apps can provide more detailed information on products, transferring them from the digital space to the real world. For example, shoppers can see how the clothes will fit their body type or configure furniture according to their room parameters. This allows customers to make their purchasing decisions with greater confidence.

Overview of JIRA

- ▶ JIRA is an **Incident Management Tool** used for Project Management, Bug Tracking, Issue Tracking and Workflow.
- ▶ Bug tracking and agile project management are allowed in JIRA and are developed by Atlassian.
 - ▶ JIRA is a platform independent tool; it can be used with any OS.
 - ▶ JIRA is multi-lingual tool – English, French, German, Japanese, Spanish, etc.
 - ▶ JIRA supports MySQL, Oracle, PostgreSQL and SQL server in the backend.
 - ▶ JIRA can be integrated with many other tools – Subversion, GIT, Clearcase, Team Foundation Software, Mercury, Concurrent Version System and many more.

Use of JIRA

Following are some of the most significant uses of JIRA.

- ▶ JIRA is used in Bugs, Issues and Change Request Tracking.
- ▶ JIRA can be used in Help desk, Support and Customer Services to create tickets and track the resolution and status of the created tickets.
- ▶ JIRA is useful in Project Management, Task Tracking and Requirement Management.
- ▶ JIRA is very useful in Workflow and Process management.

JIRA - Core Features

1. Boards

JIRA supports Scrum and Kanban boards.

These boards provide an immediate snapshot of the project to the team.

Helps to quickly review the progress of the project and see the status of the individual tasks.

Board workflow can be customized to fulfil the way a team wants to proceed

2. Business Project Template

JIRA supports **n** number of business templates to manage simple tasks and complex tasks like workflow.

Template can be customized based on the team and their approach. Ex: Workflow can be customized based on each team's approach.

Every step is accounted and team can move to achieve their goals.

3. Task Details

Tasks can be defined at the individual level to track the progress.

Status of every task, comment, attachment and due dates are stored in one place.

4. Notifications

An email can be sent for a particular task to the users.

Voting and watching features to keep an eye on the progress for the stakeholders.

Use **@mention** to get the attention of a specific team member at Comments/Description.

User will instantly notify if something is assigned or if any feedback is required.

5. Power Search

JIRA supports a powerful search functionality with Basic, Quick and Advanced features.

Use the search tool to find answers like due date, when a task was last updated, what items a team member still needs to finish.

Project information at one place, search within a project.

6. Reports

JIRA supports more than a dozen reports to track progress over a specific timeframe, deadlines, individual's contribution, etc.

Easy to understand and generate different reports those help to analyze how the team is going on.

Easy to configure these reports and display the matrices to the stakeholders.

7 Scale with Team Growth

JIRA supports any business team and any project irrespective of size and complexity.

8 Add -Ins

JIRA supports more than 100 add-ins to connect with different software to make work easy.

Wide range of add-ins makes it as universal across the globe.

9.Multilingual

JIRA supports more than 10 languages those are widely used as English (US, UK, India), French, German, Portuguese, Spanish, Korean, Japanese and Russian.

10.Mobile App

JIRA is available as a Mobile Application as well.

It is available on Google Play Store and App Store (iTunes) of Apple.

Easy to stay connected with the team while moving anywhere with notification, comments and project activity.

Asana

What Is Asana?

- ▶ Asana is similar to other task and project management software which allow teams to organize, collaborate, plan, and execute tasks.
- ▶ It acts as a perfect companion to overcome chaos and meet deadlines.
- ▶ It is a web-based task management and collaboration tool which eliminates the email mess and brings all tasks together.
- ▶ Teams can use Asana to keep track of all tasks, collaborate with other team members, exchange related files, and more.
- ▶ The tool is widely used by various famous companies like Deloitte, Airbnb, Pinterest, and Dropbox.
- ▶ Asana is becoming a favorite among industry giants due to its numerous project management and collaboration features.
- ▶ It is used by millions of individuals in 195 countries and is expanding globally.

Asana for Project Management: How Does It Work?

- ▶ Individuals and teams can use Asana to break down large work into manageable tasks.
- ▶ It's a comprehensive work management tool that allows you to track project and task progress, share files, comments, and notes, and keep track of deadlines.
- ▶ It is a modern method of working together in collaboration.
- ▶ Asana, as already mentioned above, is a task management software that has certain features like projects, workspaces, tasks, and sections to manage projects and tasks related to several clients and teams.
- ▶ Asana is mainly designed for teams to spend less time writing emails, meetings, and working on the assigned tasks.
- ▶ The Asana project management tool helps its users to work faster and take on interesting goals.

Workspaces

Workspaces are nothing but shared work areas where users can collaborate on tasks and projects. These are the primary building blocks of your Asana tool. You can create different workspaces for different clients or teams separately.

Projects

After workspaces, you will get a space where you can see a series of projects. They are your organization's main units, which can be both private and public. You can color-code them and filter them in your project section based on tags, assigned people, and priority.

Tasks

This level of work organization in Asana is to-dos. You may create tasks, fill them with a variety of information, and keep your workload balanced.

Calendar

The calendar section in Asana allows you to view all your pending tasks. You can quickly switch between 'list view' and 'calendar view' on the Asana calendar to see which tasks are due and when.

Advantages

The following are the pros of Asana:

- The design of the interface of Asana is attractive
- An easy to use calendar synced with all of your tasks
- Possible to view your personal tasks in a single place
- Email integration

Disadvantages

The following are the cons of Asana:

- There is no simple method to keep track of all items across several projects that are due on the same day or week.
- Since this application is mainly text-based, it's common for the structure to be broken, a job to be removed, or the task's name to be changed.
- It does not allow you to assign multiple people to the same task.
- With so many duties in one location, the process might get slow.
- Because Asana lacks time-tracking capabilities, you'll need to use an external time-tracking program.